



DEPARTMENT OF
COMPUTER SCIENCE

CARLOS MIGUEL GOMES FIGUEIREDO BARATA

BSc in Science and Computer Engineering

INFORMING THE TRAINING OF CONVOLUTIONAL NEURAL NETWORKS WITH FORMAL DOMAIN KNOWLEDGE

MASTER'S DISSERTATION

MASTER IN STUDY PROGRAM NAME

NOVA University Lisbon

Draft: February 10, 2026

INFORMING THE TRAINING OF CONVOLUTIONAL NEURAL NETWORKS WITH FORMAL DOMAIN KNOWLEDGE

MASTER'S DISSERTATION

CARLOS MIGUEL GOMES FIGUEIREDO BARATA

BSc in Science and Computer Engineering

Adviser: Ricardo Gonçalves

Assistant Professor, NOVA University Lisbon

Co-advisers: Matthias Knorr

Associate Professor, NOVA University Lisbon

Ludwig Krippahl

NOVA LINES

MASTER IN STUDY PROGRAM NAME

NOVA University Lisbon

Draft: February 10, 2026

ABSTRACT

Neural networks have become the leading approach for tackling problems involving unstructured data, such as image recognition, speech processing, and natural language understanding. Despite their effectiveness, these models are often considered “black boxes” because their inner workings are difficult for humans to interpret, making it challenging to explain or justify their outputs—an issue especially critical in applications that affect human lives. In contrast, symbolic AI systems inherently offer interpretability, often providing transparent reasoning or formal proofs for their conclusions. However, they struggle with tasks where neural networks perform best, such as processing visual data. Neural-symbolic approaches aim at integrating neural networks and symbolic systems. Recently, some approaches have been proposed to inform the training of a convolutional neural network with formal domain knowledge, namely by mapping neurons of the network to concepts of an ontology in the domain of the classification problem. Although some positive results have been obtained, limitations of the chosen dataset, in particular on the expressiveness of the associated ontology, did not allow drawing stronger conclusions. In this work we aim to systematically investigate the impact of incorporating formal knowledge into a neural network during the training phase by varying the amount and expressivity of injected knowledge, and evaluating its effects under multiple performance and training-related metrics.

Keywords: Artificial Intelligence · Machine Learning · Neuro-symbolic AI · Deep Learning and Reasoning · Explainable AI

RESUMO

Redes neuronais tornaram-se a abordagem dominante de lidar com problemas que envolvam dados não estruturados, como reconhecimento de imagem, processamento de fala, e compreensão de linguagens naturais. Apesar da sua eficiência, estes modelos são considerados de "caixas pretas", uma vez que o seu funcionamento interno é muito difícil de interpretar para os humanos, o que complica a explicação ou justificação das suas decisões — um problema particularmente crítico em aplicações que têm impacto direto na vida das pessoas. Em contraste, os sistemas de inteligência artificial simbólica oferecem, de forma intrínseca, um elevado grau de interpretabilidade, fornecendo frequentemente raciocínios transparentes ou provas formais para as suas conclusões. No entanto, estes sistemas tem muitas dificuldades em tarefas nas quais as redes neuronais se destacam, como o processamento de dados visuais. As abordagens neuro-simbólicas visam integrar redes neuronais e sistemas simbólicos, combinando as vantagens de ambos os paradigmas. Recentemente, têm sido propostas algumas abordagens que procuram informar o treino de redes neuronais convolucionais com conhecimento formal do domínio, nomeadamente através da associação de neurónios da rede a conceitos de uma ontologia relevante para o problema de classificação. Embora tenham sido obtidos alguns resultados positivos, limitações do conjunto de dados utilizado — em particular no que respeita à expressividade da ontologia associada — impediram a obtenção de conclusões mais robustas. Neste trabalho nós vamos investigar o impacto que incorporar conhecimento formal numa rede neuronal tem no treino da mesma, variando a quantidade de conhecimento injetado e avaliando os seus efeitos através de múltiplas métricas de desempenho e de treino.

Palavras-chave: Inteligência Artificial · Aprendizagem Automática · IA neurosimbólica · AI explicável

CONTENTS

1	Introduction	1
1.1	Context and Motivation	1
1.2	Problem Statement	2
1.3	Proposed Approach	3
1.4	Expected Contributions	3
1.5	Document Structure	4
2	Background	5
2.1	Machine Learning	5
2.1.1	Kinds of machine learning problems	5
2.2	Neural Networks	6
2.3	Convolutional Neural Networks	8
2.4	Description Logic	9
2.4.1	$\mathcal{ALC}$ Concept Descriptions	10
2.4.2	$\mathcal{ALC}$ Domain knowledge	11
2.5	Real Logic	12
2.6	Neural-Symbolic Learning	13
2.6.1	Evaluation Metrics for Neural-Symbolic Learning	14
3	State of the art	16
3.1	Statistical Relational Learning	16
3.2	Neural-Symbolic Integration Approaches	17
3.2.1	Integration Paradigms in Neural-Symbolic Learning	17
3.3	Logic Regularization and Semantic Losses	18
3.3.1	A Semantic Loss Function for Deep Learning with Symbolic Knowledge	19
3.3.2	Neural Probabilistic Soft Logic (NeuPSL)	20
3.3.3	Logic Tensor Networks (LTNs)	21
3.4	Pre-training techniques for Neural-Networks	23

3.4.1	Pre-trained Networks as Implicit Knowledge Sources	23
3.4.2	Impact of Prior Knowledge on Neural Training	23
3.5	Concept Mapping	24
3.6	Explanations of neural networks with LTNs	25
3.6.1	Model Architecture	26
3.6.2	Structure of the Model Output	26
3.6.3	Training with Logical Constraints	26
3.6.4	Evaluation methodology	27
3.7	Summary and Method Comparison	28
4	Methodology	30
4.1	Goals and approach	30
4.2	Dataset Selection	30
4.3	Architecture	31
4.4	Evaluation metrics	32
4.5	Experiments	32
4.6	Task Schedule	33
	Bibliography	35

LIST OF FIGURES

2.1	Architecture of a neural network	7
2.2	Architecture of a convolutional neural network	9
3.1	NeuPSL inference and learning pipeline	20
3.2	Model output: each number inside the circle indicates the position of the neuron in the result array passed to the predicate. The first two neurons represent the final class of the image, while the last four correspond to intermediate concepts of the dataset. Reproduced from [29].	27

LIST OF TABLES

4.1	Planned thesis timeline	34
-----	-----------------------------------	----

INTRODUCTION

This chapter introduces an overview of the context and motivation of this thesis, detailing the goals and the approach taken to address the problem.

1.1 Context and Motivation

Neural networks (NNs) [1] have emerged as a transformative force across various sectors, revolutionizing industries such as healthcare [25], finance [21], and automotive technology [27]. Their use became even more widespread with the recent popularization of Large Language Models [8], to the point that a large portion of the working-age population regularly interacts with these models, either for work or leisure. As the number of people interacting with NNs increases rapidly, it is now more important than ever that the models give reliable and unbiased answers.

Neural Networks belong to the class of *sub-symbolic systems*, a family of artificial intelligence approaches in which knowledge is represented implicitly through distributed numerical parameters rather than explicit symbolic structures. These systems have demonstrated remarkable performance across a wide range of tasks; however, they suffer from a significant limitation: it is often extremely difficult for humans to understand how a network arrives at a given decision. This opacity arises from the large number of parameters and computations involved in processing each input, which can lead to a justified lack of trust in their outputs [20].

In other words, sub-symbolic systems typically lack **interpretability** (the ability for a human to inspect the internal structure of the model and understand how it operates) and **explainability** (the ability of the model to provide understandable justifications for its decisions). These properties, by contrast, are characteristic of *symbolic systems*, which rely on explicit, structured representations of knowledge and perform reasoning through deductive processes.

To mitigate these limitations, researchers have explored methods for improving the interpretability of sub-symbolic systems. This research led to the start of development of the field of Explainable Artificial Intelligence (XAI), that aims to improve simultaneously

the interpretability and explainability of AI systems. One of the most important new technologies developed are Neuro-Symbolic (NeSy) approaches [4], that aim to combine the traditional symbolic reasoning, like an ontology, with modern neural networks. These hybrid systems' goal is to have the learning capabilities of neural models while retaining some transparency from symbolic AI.

In recent years, several NeSy approaches have been investigated, including the work presented in [29]. In that work, the researchers aimed to investigate if NeSy approaches can be used to increase the explainability of NNs, by incorporating formal knowledge about the dataset's domain into the network during its training. To make NNs more explainable, the goal of the researchers was to investigate if it was possible to develop a way that the network would not only classify the image correctly, but also identify the intermediary concepts present in that image. For example, if the task was to identify if the image contained a cat, an intermediate concept could be the fact that it has whiskers, or a tail. Using the framework Logic Tensor Networks (LTNs) [32], they mapped each of the relevant intermediate concepts to a neuron of the network, which would activate whenever the image identified that specific concept. Using this technique, the researchers were able to train the network to accurately identify both the target class and the intermediate concepts in certain cases, particularly when the injected knowledge was relatively simple. However, when more complex or extensive knowledge was introduced, the approach struggled to accurately identify the corresponding concepts.

Overall, this line of research demonstrated that LTNs can successfully be used to incorporate formal knowledge into neural networks and promote the emergence of semantically meaningful intermediate concepts, thereby improving model explainability.

1.2 Problem Statement

Although past research was able to achieve positive results, the impact on other aspects of training when incorporating formal knowledge into the network has yet to be explored. In particular, existing work does not evaluate how the integration of formal knowledge affects core performance-related metrics, such as classification accuracy, convergence behavior, training stability, or generalization capability. This omission represents a significant gap in literature, as neural networks are typically deployed in scenarios where performance and reliability are as critical as explainability. Without understanding these trade-offs, it is unclear whether these methods can be safely and effectively adopted in practical applications. In particular, it is unknown whether improving explainability comes at the cost of reduced predictive performance, increased training complexity, or altered learning dynamics.

This thesis systematically investigates the effects of integrating formal knowledge into neural network training beyond explainability. Specifically, we pretend to analyze how the integration of that formal knowledge influences performance-related metrics such as accuracy, convergence speed, and generalization, compared to standard neural network

training. By doing so, this work aims to clarify whether formal knowledge integration introduces trade-offs or synergies between explainability and performance.

1.3 Proposed Approach

To address the impact of formal knowledge on neural network training, we adopt a neural-symbolic learning framework in which domain knowledge about the dataset is explicitly formalized using an ontology. This ontology encodes relevant concepts and relations present in the data and serves as the basis for injecting symbolic knowledge into the learning process.

To conduct this investigation, we will develop multiple networks for an image classification task. First, a baseline model is trained using images from the selected dataset. The other models are going to be developed using the same architecture and dataset, but will also have logical constraints that were integrated using a neural-symbolic technique. These models are then compared to measure the impact of formal knowledge integration.

The experimental evaluation focuses on measuring the impact of formal knowledge on multiple training-related metrics beyond explainability. In particular, we analyze sample efficiency by evaluating whether the knowledge-informed network requires fewer training examples to achieve performance comparable to the baseline model. Additionally, we assess predictive performance by comparing classification accuracy when both models are trained on the same amount of data.

We further study training dynamics, including convergence speed, by measuring the number of training iterations required for each model to reach predefined accuracy thresholds. To better understand the role of symbolic structure, we conduct additional experiments using different subsets of the ontology, enabling the analysis of how various types of conceptual and relational knowledge influence learning behavior and performance of the model.

To conduct the proposed experiments, we employ LTNs [32] as the primary mechanism for integrating formal knowledge into neural network training. The symbolic knowledge is provided through an ontology derived from the VCB dataset, originally proposed in the work [30], which is adapted and extended in this thesis to better align with the experimental objectives.

1.4 Expected Contributions

We expect the main contributions of this research to be:

- A systematic empirical evaluation of the impact of formal knowledge integration on neural network training, considering performance-related metrics such as accuracy, convergence speed, and sample efficiency, in addition to explainability.

- An analysis of the trade-offs and potential synergies between explainability and predictive performance in neural-symbolic learning frameworks.
- An analysis of the trade-offs between convergence speed and constraint satisfaction of models using LTNs.
- A reproducible experimental methodology for comparing knowledge-informed and standard neural networks under identical architectural and training conditions.

1.5 Document Structure

The remaining of this document is structured as follows:

Chapter 2: Background introduces the foundational concepts required to understand the rest of the thesis. It provides an overview of machine learning and neural networks, followed by a discussion of logic-based formalisms and neural-symbolic learning.

Chapter 3: State of the Art reviews existing approaches in explainable AI and neural-symbolic learning, with particular emphasis on methods that integrate formal knowledge into neural network training.

Chapter 4: Methodology presents the proposed approach, detailing the model architecture, training procedure, and other information relevant to subsequent stages of the thesis development.

BACKGROUND

This chapter presents background knowledge relevant to the understanding of this dissertation. Before diving deeper into NeSy approaches, we will explore some relevant subjects, such as Neural Networks and Description Logic, to facilitate understanding of the subsequent chapters.

2.1 Machine Learning

Machine learning [2] is a field of research in artificial intelligence concerned with the development and study of statistical algorithms that can learn from data and generalise to unseen data, thereby perform tasks without explicit instructions.

Machine learning constitutes a central component of the rapidly expanding field of data science. By employing statistical methods, these algorithms can be trained to perform classification and prediction tasks or to extract meaningful patterns and insights in data mining applications. These insights subsequently inform decision-making processes in both scientific and industrial contexts, with the goal of positively influencing key performance and growth metrics. These machine learning models are typically developed using established frameworks such as TensorFlow[7] and PyTorch[14].

Machine learning algorithms comprise a decision process for generating predictions or classifications, an error (or loss) function used to evaluate model performance, and an optimization procedure that iteratively adjusts model parameters to minimize prediction error and improve the fit to the training data, until a specified accuracy or convergence criterion is achieved.

2.1.1 Kinds of machine learning problems

There are three common types of machine learning problems:

- **Supervised Learning:** Supervised learning describes tasks in which a dataset that contains both labels and features is provided, and the goal is to create a model that predicts the correct label based on the input features. Supervised Learning tasks

can be divided into multiple categories, but the most common ones are regression tasks, which are used to approximate a continuous value, and classification tasks, which are used when the output has a discrete amount of possible results. Image classification tasks, including the one developed in this thesis, as well as broader computer vision problems, constitute prominent applications of supervised machine learning.

- **Unsupervised and Self-Supervised Learning:** Unsupervised learning refers to tasks where the models are trained on unlabeled data, with the goal of discovering underlying structures or patterns, such as clusters. Methods such as k-means clustering [22] and Principal Component Analysis (PCA) [10] are commonly used for feature extraction[36] or data visualization tasks [11]. Self-supervised learning is usually considered a subfield of this category. These types of tasks follow a paradigm in which supervisory signals are automatically derived from the data itself, allowing models to learn meaningful representations without requiring manual annotation. These method has gained a lot of attention in the last few years, but it still not as commonly used as the other three.
- **Reinforcement Learning:** Reinforcement Learning (RL) is a suite of techniques that allows us to build machine learning systems that take decisions sequentially. Instead of learning from pre-existing data samples, this approach learns by trial and error by reinforcing sequences of successful outcomes. Reinforcement learning has seen significant applications in domains such as finance [12] and the automotive sector [18].

2.2 Neural Networks

In machine learning, we often refer to the computational structures that transform data as models. Neural Networks (NNs) [1] describes a powerful class of models, composed of a sequence of transformations, denominated layers. NNs are often described as "universal function approximators" due to their theoretical capacity to approximate a wide range of functions. Throughout this thesis, we will mostly use the definitions from [40] when referencing machine learning concepts and will resort to the same notations.

The first layer of a NN is the input layer and the last layer is the output layer; each layer in between is called a hidden layer. Each layer consists of multiple neurons interconnected with neurons from adjacent layers. Figure 2.1 illustrates a typical neural network architecture. The output of a neuron is computed from weighted connections with neurons from the previous layer, a bias term, and a non-linear activation function. The learning process in a neural network involves two key phases: **forward propagation** and **backpropagation**.

The forward propagation is the first phase of each training iteration. The input data flows through the network, layer by layer, to generate an output. Each neuron processes its input in the following steps:

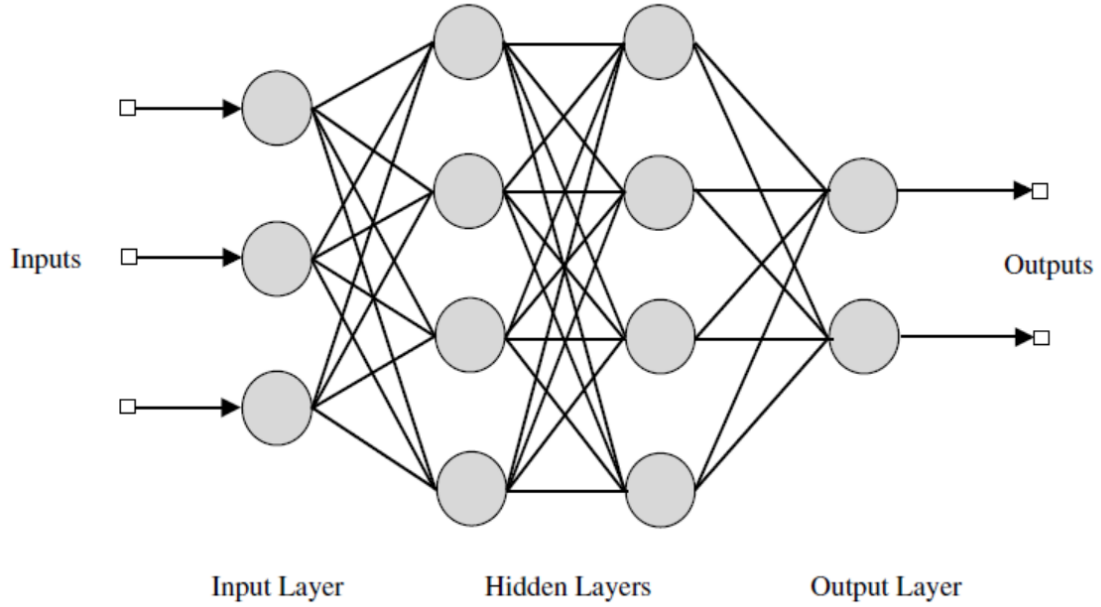


Figure 2.1: Architecture of a neural network

- **Weighted Sum:** The input values are passed through from neurons in the previous layer. Then, a weighted sum is calculated from these values, where the weights are parameters learned by the network. Additionally, a bias term is added to the sum.
- **Activation Function:** To introduce non-linearity into the model (which is necessary for solving complex problems), the neuron applies an activation function to the weighted sum. Some important examples are the sigmoid(2.1), commonly used in binary classification because its output is a continuous value between 0 and 1, and ReLU (2.2) that outputs 0 when the input is negative, and returns the input when it's not.

$$\text{Sigmoid}(x) = \sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.1)$$

$$\text{ReLU}(x) = \max(0, x) \quad (2.2)$$

After the activation, the value will be used as input for the neurons in the subsequent layer. After the forward pass is completed, the network will produce an output value and will compare it to the target value using a loss function. That is how the model quantifies the error, which is the difference between the true output and the predicted output. Two of the more widely used loss functions are Mean Square Error (2.3), that is used in regression tasks, and Cross-entropy (2.4) loss is widely used in classification tasks.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad (2.3)$$

where y_i denotes the true value and $\hat{y}_i$ denotes the predicted value.

$$\mathcal{L}_{\text{CE}} = - \sum_i y_i \log(\hat{y}_i), \quad (2.4)$$

where y_i represents the true class label and $\hat{y}_i$ represents the predicted probability for that class.

After computing the loss, the model parameters, including both weights and biases, are updated in order to minimize this loss. This process, known as backpropagation, consists of two main phases: First, the algorithm calculates the gradient of the loss function with respect to each weight in the network. These gradients quantify the sensitivity of the loss to small changes in the weights, indicating how each parameter should be adjusted to reduce the overall error. Then, the weights are updated using gradient descent, which updates the weights in the opposite direction of the gradient, moving the parameters toward a minimum of the loss function. This process of updating each weight can be written in the following way:

$$w \leftarrow w - \eta \nabla_w \mathcal{L}, \quad (2.5)$$

where w denotes a model parameter (either a weight or a bias), $\eta > 0$ is the learning rate, and $\nabla_w \mathcal{L}$ represents the gradient of the loss function $\mathcal{L}$ with respect to w .

This process of executing a forward pass, calculating the loss, and updating the parameters in the backpropagation will be executed for many iterations, usually called epochs, until the networks starts to converge. This happens when the weights are only slightly changed each iteration, and the loss function reaches its minimum.

2.3 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) [15] are a specialized class of neural networks designed to process data with a grid-like structure, such as images. Unlike traditional neural networks, CNNs exploit the spatial structure of images by learning local patterns, making them effective for visual recognition tasks. A CNN is primarily composed of three types of layers: convolutional layers, pooling layers, and fully connected layers. Figure 2.2 illustrates a typical convolutional neural network architecture, highlighting the transformations of the dimensions of the input after each layer.

Convolutional layers apply learnable filters, known as kernels, to the input data in order to extract relevant features and produce feature maps. These filters are shared across the entire input, which reduces the number of trainable parameters and allows the network to detect patterns regardless of their position in the input.

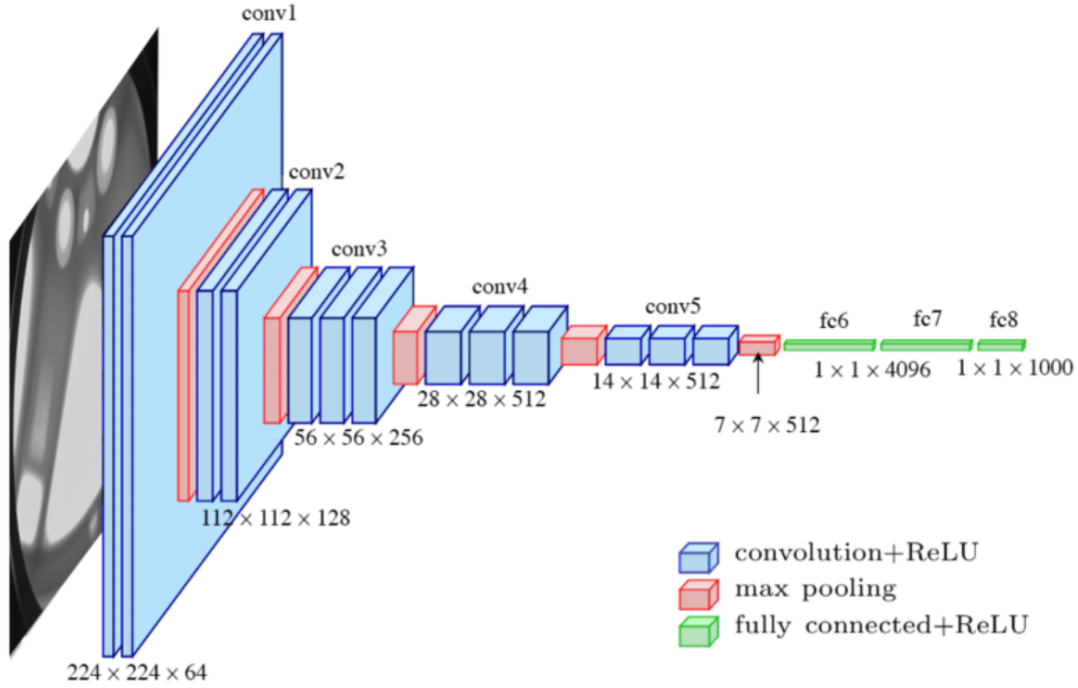


Figure 2.2: Architecture of a convolutional neural network

Pooling layers are used to reduce the spatial size of feature maps while retaining the most important information. This reduction lowers computational cost and improves robustness to small variations in the input. As additional layers are stacked, CNNs learn progressively more abstract representations, ranging from simple features such as edges to higher-level patterns.

Finally, the last part of the architecture of a CNN consists of one or more fully-connected layers. Each neuron of these layers is connected to each node from the previous layer, which enables the network to classify images based on features extracted from the whole image. Due to their efficiency and strong performance, CNNs have become the standard approach for many computer vision applications, including image classification [41], object detection [6], and image segmentation [38].

2.4 Description Logic

Description Logics (DLs) are a family of formal knowledge-representation languages designed to represent and reason about the knowledge of an application domain in a structured and formally well-understood way. The examples in this section are adapted from [3].

DLs allow the definition of logical restrictions, referred to as axioms, over the properties of a set of elements. Axioms are expressed in a formal language composed of:

- **Concepts:** They represent classes of elements, and are interpreted as unary predicates.

The set of elements that belong to each concept is called the concept's extension. For example, the concept *Teacher* could represent the set of people that are teachers.

- **Roles:** A role describes relations between elements, and are interpreted as binary relations. The set of pairs of elements that belong to each role is called the role's extension. For example, the role *Friends* could represent pairs of people who are friends.
- **Individuals:** An individual represents a specific element. They describe typical elements that must be present in any set, allowing axioms to be defined over such elements. The element that corresponds to each individual name is called its extension. For example, the individual name *John* could represent a person named John.

In Description Logic, an *interpretation* provides a formal semantics by assigning meaning to the symbols in the knowledge base. Formally, an interpretation $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$ consists of a non-empty domain $\Delta^{\mathcal{I}}$ of objects, and an interpretation function $\cdot^{\mathcal{I}}$ that maps each concept to a subset of $\Delta^{\mathcal{I}}$, each role to a subset of $\Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$, and each individual to an element of $\Delta^{\mathcal{I}}$. Intuitively, an interpretation represents one possible "world" in which the statements of the knowledge base may be evaluated as true or false. With this formal semantics in place, we can now define more complex concepts by combining atomic concepts using logical constructors, as provided by the $\mathcal{ALC}$ language.

2.4.1 $\mathcal{ALC}$ Concept Descriptions

DL axioms are constructed from *concept descriptions*, which are expressions defining sets of elements that may be more complex than a simple concept name. In this thesis, whenever we refer to a "concept," we mean an $\mathcal{ALC}$ concept description. Concept descriptions are classified into *atomic* and *compound* concepts.

Atomic concepts include:

- Individual concept names.
- The universal concept $\top$, which applies to all elements.
- The empty concept $\perp$, which applies to no elements.

Given two concepts (atomic or compound) C and D , and a role name R , $\mathcal{ALC}$ allows the construction of the following *compound concepts*:

- $\neg C$ (Negation), representing all elements that do **not** belong to C .
- $C \sqcap D$ (Conjunction), representing all elements that belong to both C and D .
- $C \sqcup D$ (Disjunction), representing all elements that belong to either C or D .

- $\forall R.C$ (Value restriction), representing all elements for which *all* R -related elements belong to C . Elements with no R relations also satisfy this description.
- $\exists R.C$ (Existential restriction), representing all elements for which there exists *at least one* R -related element that belongs to C .

2.4.2 $\mathcal{ALC}$ Domain knowledge

The syntax of DLs includes base concepts and roles, with knowledge bases divided into a TBox (terminological axioms) and an ABox (assertional axioms). TBox statements define general properties of concepts and roles, while ABox assertions specify facts about individuals. For example, TBox statements capturing knowledge about a university domain could include *a teacher is a person who teaches a course*, *a student is a person who attends a course* and *students do not teach*, while ABox statements might include *John is a teacher*, *Machine Learning (ML) is a course* and *John teaches ML*. One crucial feature of DLs is that these statements can be translated using a formal, logic-based semantics. The above statements can be rendered as sentences in first-order logic (FOL) as follows:

$$\begin{aligned}
& \forall x \left(\text{Teacher}(x) \leftrightarrow (\text{Person}(x) \wedge \exists y (\text{Course}(y) \wedge \text{Teaches}(x, y))) \right), \\
& \forall x \left(\text{Student}(x) \leftrightarrow (\text{Person}(x) \wedge \exists y (\text{Course}(y) \wedge \text{Attends}(x, y))) \right), \\
& \forall x \left((\exists y \text{ teaches}(x, y)) \rightarrow \neg \text{Student}(x) \right), \\
& \text{Teacher}(\text{John}), \\
& \text{Course}(\text{ML}), \\
& \text{Teaches}(\text{John}, \text{ML}).
\end{aligned}$$

These statements can also be written using the DL syntax:

$$\begin{aligned}
& \text{Teacher} \equiv \text{Person} \sqcap \exists \text{teaches}.\text{Course} \\
& \text{Student} \equiv \text{Person} \sqcap \exists \text{attends}.\text{Course} \\
& \exists \text{teaches}.\top \sqsubseteq \neg \text{Student} \\
& \text{John} : \text{Person} \\
& \text{ML} : \text{Course} \\
& (\text{John}, \text{ML}) : \text{teaches}
\end{aligned}$$

In both of these examples, the first three sentences constitute the TBox, and the last three the ABox.

In summary, Description Logics provide a formal, structured, and decidable framework for representing knowledge about a domain, allowing the definition of both general concept relationships (TBox) and individual-specific facts (ABox). By translating DL statements

into first-order logic, we can formally reason about the domain, check for consistency, and verify concept hierarchies or instance membership. This foundation is particularly important for neural-symbolic integration, as it allows ontologies to be systematically encoded into neural networks, providing prior knowledge that can guide learning and improve both efficiency and explainability.

2.5 Real Logic

However, while DLs offer a powerful symbolic framework for representing structured domain knowledge, their classical semantics relies on a binary notion of truth that is not directly compatible with gradient-based learning. To enable the integration of such knowledge into neural networks, a continuous and differentiable semantics is required. Real Logic provides this extension by interpreting logical formulas with real-valued truth degrees, forming the foundation of neural-symbolic approaches such as Logic Tensor Networks and related neural-symbolic architectures.

We start from a first-order language $\mathcal{L}$, whose signature contains a set $\mathcal{C}$ of constant symbols, a set $\mathcal{F}$ of function symbols, and a set $\mathcal{P}$ of predicate symbols. The sentences of $\mathcal{L}$ are used to express relational knowledge. For example, the atomic formula $R(o_1, o_2)$ states that objects o_1 and o_2 are related through the binary relation R ; the formula

$$\forall x \forall y (R(x, y) \rightarrow R(y, x))$$

states that R is a symmetric relation, where x and y are variables; and the formula

$$\exists y R(o_1, y)$$

states that there exists an (unknown) object that is related to object o_1 through R . For simplicity, LTNs assume that logical formulas are expressed in a normal form that allows them to be uniformly translated into differentiable constraints. In particular, existential quantifiers are eliminated via Skolemization, and formulas are reduced to implications or clauses, which can be directly grounded and optimized during learning.

As for the semantics of $\mathcal{L}$, we deviate from the standard abstract semantics of First Order Logic and propose a concrete semantics with sentences interpreted as tuples of real numbers. To emphasize the fact that $\mathcal{L}$ is interpreted in a "real" world, we use the term (*semantic*) *grounding*, denoted by $\mathcal{G}$, instead of the more standard notion of *interpretation*.

- $\mathcal{G}$ associates an n -tuple of real numbers $\mathcal{G}(t) \in \mathbb{R}^n$ to each closed term t of $\mathcal{L}$; intuitively, $\mathcal{G}(t)$ represents the set of numeric features of the object denoted by t .
- $\mathcal{G}$ associates a real number in the interval $[0, 1]$ to each clause φ of $\mathcal{L}$; intuitively, $\mathcal{G}(\varphi)$ represents the degree of confidence in the truth of φ , with higher values corresponding to higher confidence.

Given this grounding $\mathcal{G}$, the truth value of complex formulas is computed by recursively combining the groundings of their constituent atoms according to the chosen fuzzy logical operators. Logical connectives such as conjunction, disjunction, and implication are therefore interpreted through continuous functions, typically based on t -norms and their associated operators. This choice ensures that the evaluation of logical formulas remains differentiable with respect to the grounding parameters.

Common t -norms and their respective t -conorms include:

- **Minimum t -norm:**

$$T_{\min}(a, b) = \min(a, b), \quad S_{\max}(a, b) = \max(a, b)$$

- **Product t -norm:**

$$T_{\prod}(a, b) = ab, \quad S_{\text{prob}}(a, b) = a + b - ab$$

- **Łukasiewicz t -norm:**

$$T_{\text{Ł}}(a, b) = \max(0, a + b - 1), \quad S_{\text{Ł}}(a, b) = \min(1, a + b)$$

As a result, each grounded clause is assigned a real-valued degree of satisfaction in the interval $[0, 1]$, rather than being simply true or false. This continuous notion of truth enables LTNs to reason in the presence of uncertainty and noise, while also allowing logical knowledge to be seamlessly integrated into gradient-based learning procedures.

2.6 Neural-Symbolic Learning

Neural-symbolic (NeSy) learning aims to combine the representational power and learning capabilities of neural networks with the structured, interpretable reasoning mechanisms of symbolic logic. While neural models excel at learning from raw, high-dimensional data such as images, they typically lack explicit mechanisms for incorporating prior knowledge or enforcing logical consistency. Symbolic approaches, on the other hand, offer formal semantics and strong reasoning guarantees but struggle to scale to noisy, perceptual data. Neural-symbolic systems seek to overcome these limitations by integrating logical knowledge directly into the learning process of neural networks, allowing both paradigms to complement each other.

Within this framework, formal knowledge, often expressed through Description Logics or First-Order Logic, is translated into differentiable constraints that can be optimized jointly with standard learning objectives. Approaches including, but not limited to, LTNs leverage Real Logic to associate logical formulas with continuous truth values, enabling gradient-based optimization while preserving semantic interpretability. As a result, neural-symbolic models are not only trained to fit data, but also to satisfy domain-specific constraints, encouraging the emergence of structured and semantically meaningful representations.

2.6.1 Evaluation Metrics for Neural-Symbolic Learning

Evaluating neural-symbolic models requires more than standard predictive performance measures. In addition to assessing how accurately a model fits the observed data, it is necessary to evaluate how well its predictions comply with the formal knowledge encoded in logical constraints and ontologies. For this reason, neural-symbolic learning frameworks typically rely on a combination of classical machine learning metrics and knowledge-aware metrics. This section introduces the evaluation criteria commonly used in neural-symbolic systems, which will later be instantiated and applied in the experimental evaluation of this thesis.

Classification Accuracy and F1-Score. We use accuracy and F1-score as primary measures of predictive performance. Accuracy captures the proportion of correctly classified building images, providing a simple and intuitive measure of overall performance. However, since some categories may be under-represented in the dataset, accuracy alone may be misleading. For this reason, we also report the F1-score, which balances precision and recall and therefore provides a more informative assessment when class distributions are imbalanced. These metrics allow us to compare models in terms of their ability to correctly recognize visual categories from images.

Sample Efficiency Sample efficiency measures how effectively a model learns from limited amounts of training data, and is particularly important in domains where data collection is costly or constrained. In neural-symbolic learning, incorporating prior formal knowledge is often hypothesized to improve sample efficiency by introducing inductive biases that constrain the hypothesis space and guide optimization. Consequently, sample efficiency is commonly evaluated by comparing model performance across varying training set sizes.

Training Dynamics Training dynamics describe how a model’s performance evolves during learning, including aspects such as convergence speed and stability. In the context of neural-symbolic models, training dynamics are of particular interest because symbolic constraints may influence the optimization landscape, potentially accelerating convergence or improving stability. These properties are commonly analyzed by tracking performance metrics across training iterations.

Constraint Satisfaction Because this thesis focuses on learning under formal knowledge, we additionally measure how well the trained models satisfy the logical constraints imposed by the ontology. In the LTN framework, each logical formula is associated with a degree of satisfaction in the interval $[0, 1]$. We therefore report the average satisfaction level over all grounded clauses in the test set. This metric directly quantifies how consistent the model’s predictions are with the encoded symbolic knowledge, independently of classification accuracy.

Semantic Consistency To complement clause-level satisfaction, we also evaluate semantic consistency at the prediction level. This metric measures the proportion of predictions that violate at least one logical constraint. This metric provides an intuitive interpretation of how often the neural network produces logically invalid outputs.

Together, these metrics allow us to evaluate both sides of the neural–symbolic trade-off. Accuracy and F1-score quantify how well the model fits the visual data, while constraint satisfaction and semantic consistency quantify how well it respects the formal knowledge encoded in the ontology. Training dynamics and sample efficiency will measure training characteristics of the model. The aggregation of these metrics is essential for assessing whether neural–symbolic integration actually improves not only the prediction quality and the logical reliability of the model, but also how relevant these systems are as alternative to classic deep learning models.

STATE OF THE ART

In this chapter introduces multiple NeSy approaches, as well as some important concepts to better understand the systems discussed. We begin by introducing a classification framework that highlights the main characteristics of neural-symbolic approaches. This structure provides a foundation for understanding how design choices shape the interaction between symbolic knowledge and neural computation, and how these perspectives relate to the objectives of this thesis. We then provide a more detailed overview of each approach, examining its underlying principles and assessing its relevance and applicability to the objectives of this work.

3.1 Statistical Relational Learning

Statistical Relational Learning(SRL) [16] is a subfield of machine learning that integrates probabilistic reasoning with relational and logical representations. Its main purpose is to develop learning models that handle uncertain information extracted from real world scenarios, and produce structured representations that describe objects, attributes and their relations.

Relational structures enable the modeling of objects through their attributes and relationships. However, traditional relational learning systems often struggle with noisy, duplicate, or missing data due to their deterministic representations of the domain. SRL addresses this limitation by combining probabilistic reasoning with relational representations, allowing models to handle uncertainty while still capturing the rich structure of the domain. By integrating probabilities with logic, SRL can effectively reason about incomplete, ambiguous, or conflicting information, making it well-suited for real-world applications where data is imperfect or relationally complex. Representative SRL approaches include Inductive Logic Programming [17] and Probabilistic Soft Logic (PSL) [5].

While SRL provides a principled framework for reasoning under uncertainty in relational domains, its primary focus is on integrating logical representations with probabilistic models, often without relying on deep neural architectures. In contrast, NeSy approaches

emphasize the integration of symbolic knowledge with neural networks, enabling joint representation learning and reasoning through differentiable or hybrid models. Although these paradigms overlap and share common foundations, this dissertation focuses on NeSy methods, as they allow explicit investigation of how symbolic knowledge influences neural training dynamics. Consequently, SRL is presented as foundational background, while the subsequent sections concentrate on neural-symbolic knowledge-infusing approaches.

3.2 Neural-Symbolic Integration Approaches

In the following sections, several representative NeSy approaches are reviewed, each illustrating different design choices for integrating symbolic knowledge with neural computation. Rather than prioritizing a specific framework at the outset, this section focuses on identifying and contrasting the key characteristics of these approaches and their implications for learning and reasoning. To structure this analysis, the following subsection introduces a set of shared dimensions that serve as a conceptual lens for evaluating neural-symbolic methods throughout the chapter.

3.2.1 Integration Paradigms in Neural-Symbolic Learning

The classification used in this thesis is based on the survey presented in [28]. They were originally proposed to classify StarAI approaches [24], that is, the area of AI that focuses on statistical relational reasoning, and it was adapted for NeSy approaches by the researchers, with the reasoning that many of the challenges and solutions that are encountered in the development on StarAI systems are also found on the development of NeSy systems. The survey presents slightly different definitions of each dimension for starAI and NeSy. As this thesis focuses on NeSy approaches, only the corresponding definitions are presented.

- **directed vs undirected models:** NeSy approaches can be distinguished by whether they emulate directed logical inference (e.g., proof-based, rule-driven reasoning) or treat logical knowledge as undirected soft constraints on the behavior of a predictive model.
- **grounding vs proof based inference:** Proof-based approaches derive predictions from the enumeration of logical proofs, whose contributions can be learned and combined, as in DeepProbLog [19]. In contrast, grounding-based approaches, such as LTNs, operate over groundings of the logical theory and learn shared parameters by optimizing over the resulting ground model.
- **integrating logic with probability and/or neural computation:** This dimension captures which paradigm acts as the primary modeling framework. Most NeSy approaches adopt neural networks as the core component and incorporate logic or probability as auxiliary constraints, while others extend existing symbolic or probabilistic logic frameworks to include neural components.

- **Logical semantics:** NeSy systems differ in the semantics they adopt, ranging from classical Boolean semantics to probabilistic and fuzzy semantics with continuous truth values. For example, DeepProbLog [19] relies on probabilistic semantics, whereas LTNs employ fuzzy logic to model degrees of truth.
- **Learning parameters or structure:** This dimension concerns whether the approach learns only numerical parameters or also induces logical rules or model structure from data.
- **Symbolic vs. sub-symbolic entity representation:** Approaches differ in whether entities are represented as discrete symbols or as continuous embeddings learned from data.
- **Type of logic used:** NeSy systems may rely on different logical formalisms, such as first-order logic, description logics, or their differentiable relaxations.

These dimensions are not intended to support an exhaustive or tabular comparison of all neural-symbolic approaches discussed in this chapter. Instead, they provide a shared conceptual lens that informs the qualitative analysis of each method and the criteria used to assess its suitability for the objectives of this thesis. In particular, they help clarify how and where symbolic knowledge interacts with neural computation, whether such interaction directly influences learning dynamics or is confined to inference or output regularization, and to what extent internal representations are shaped by formal constraints. Throughout the remainder of this chapter, each approach is discussed with respect to the most relevant of these dimensions, and its limitations are motivated accordingly, rather than through an explicit dimension-by-dimension comparison.

3.3 Logic Regularization and Semantic Losses

An increasingly common approach in neural-symbolic learning consists of incorporating formal knowledge into neural networks by translating logical constraints into differentiable loss functions. Rather than enforcing logic as hard constraints, these methods use logic as a form of *regularization*, guiding the learning process toward solutions that are consistent with prior knowledge while remaining compatible with data-driven optimization.

This paradigm, often referred to as *logic regularization*, treats logical rules as soft constraints whose violations are penalized during training. By integrating symbolic knowledge directly into the objective function, logic regularization provides a flexible mechanism for injecting structure, improving generalization, and reducing data requirements.

3.3.1 A Semantic Loss Function for Deep Learning with Symbolic Knowledge

A representative example of logic regularization is the Semantic Loss framework proposed by Xu et al. [37], which introduces logical constraints directly as differentiable loss terms. Another important example is the aforementioned LTNs.

Semantic loss measures the degree to which the probabilistic outputs of a neural network satisfy a given logical formula. Instead of simulating symbolic reasoning or relaxing logical operators through fuzzy logic, this approach defines a loss function directly from how well the outputs match a given constraint. As a result, this function precisely captures the meaning of the constraint, and is independent of its syntax.

By incorporating semantic loss as an additional training objective, logical constraints act as soft regularizers that guide learning toward outputs that respect the structure of the problem. This makes it possible to exploit symbolic knowledge even in settings with limited labeled data, as the constraints themselves provide a learning signal over unlabeled examples.

In the article, the researchers defined semantic loss $L_s(\alpha, \mathbf{p})$ as a function of a sentence α in propositional logic, defined over variables $X = \{X_1, \dots, X_n\}$, and a vector of probabilities $\mathbf{p}$ for the same variables X . Element $\mathbf{p}_i$ denotes the predicted probability of variable X_i and corresponds to a single output of the neural network.

$$L_s(\alpha, \mathbf{p}) \propto -\log \sum_{x \models \alpha} \left(\prod_{i: x \models X_i} p_i \prod_{i: x \models \neg X_i} (1 - p_i) \right).$$

As an example, if α is "exactly one variable is true", $X = \{X_1, X_2, X_3\}$ and $\mathbf{p} = (0.7, 0.2, 0.1)$, the 3 possible states x of α are (1,0,0), (0,1,0) and (0,0,1). For each possible state of α , the contribution is computed as the product of the probabilities of variables assigned true and the complements of the probabilities of variables assigned false. In this example, for the state (1,0,0) the product is

$$0.7 * (1 - 0.2) * (1 - 0.1) = 0.7 * 0.8 * 0.9 = 0.504$$

The semantic loss is computed by calculating the sum of the contributions of all satisfying assignments and applying the negative logarithm to the resulting probability mass. In this example, the total sum for each state is 0.504, and the loss is equal to about 0.54, which means that only about 54% of the probability mass satisfies the constraint, so the network will be penalized.

Despite its strengths, semantic loss is limited to propositional constraints over the output variables of a neural network and does not support relational reasoning, first-order quantification, or the modeling of latent symbolic structures. As a result, it is primarily suited for enforcing structural properties of predictions rather than capturing rich relational knowledge. It also only operates at the output level, which leads to a decreased impact on other areas of the NN. From the perspective of this thesis, semantic

loss represents a related and interesting technique, but it is not relevant to utilize as a comparison with other NeSy approaches.

3.3.2 Neural Probabilistic Soft Logic (NeuPSL)

Neural Probabilistic Soft Logic (NeuPSL)[26] is a neural extension of the Probabilistic Soft Logic (PSL) framework [5]. PSL enables reasoning over uncertain, relational, and structured data using weighted first-order logic formulas. Unlike traditional logical systems, where predicates are either true or false, PSL uses *soft truth values*, meaning that it assigns each predicate a real-value truth score in the interval between 0 and 1. Logical connectives such as conjunctions or disjunctions are relaxed using continuous t-norm semantics, enabling smooth optimization.

NeuPSL extends PSL to combine neural perception with symbolic reasoning for scalable joint learning and inference. The key innovation of this framework is the new kind of predicates, the **neural predicates** whose outputs are produced by neural networks. Neural predicates preserve PSL semantics while adding low-level neural perception. It is based on a mathematical framework Neuro-Symbolic Energy-Based Models (NeSy-EBMs), that was also proposed in the same article. This framework enables many NeSy systems to utilize established Energy-Based Model learning losses and algorithms. Figure 3.1 provides a graphical representation of this process. The symbolic potentials and neural networks together define a *deep hinge-loss Markov random field* (Deep-HL-MRF), a probabilistic graphical model that supports scalable convex joint inference.

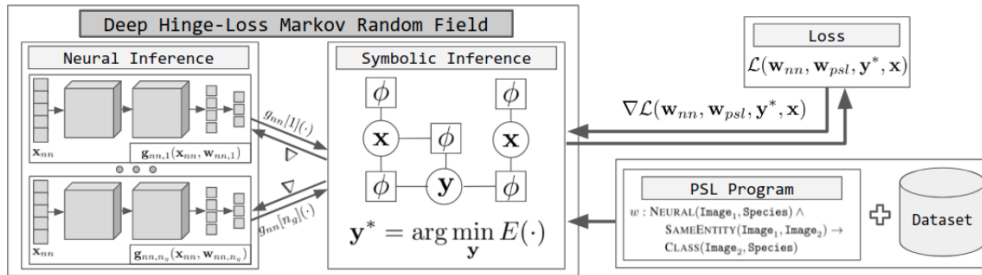


Figure 3.1: NeuPSL inference and learning pipeline

NeuPSL instantiates the symbolic potentials of its energy function using the PSL language where *atoms* are the dependencies between relations and attributes of entities, and are encoded with weighted first-order logical clauses and linear arithmetic inequalities into *rules*. **Neural inference** corresponds to the forward computation performed by the neural components of the model. Given an input, each neural network produces predictions that serve as soft evidence for the symbolic layer. These predictions capture data-driven patterns and provide initial estimates for the truth values of relevant variables. **Symbolic inference**, on the other hand, consists of jointly reasoning over the neural

predictions and the symbolic rules encoded in the PSL model. During this process, the framework adjusts the output variables in order to satisfy the logical constraints as much as possible, while remaining consistent with the neural outputs. NeuPSL learns by finding both neural and symbolic weights that assign low energy to the correct values of the output variables and higher energies to incorrect ones.

While NeuPSL enables tight integration between symbolic reasoning and neural perception, the symbolic knowledge primarily influences inference over output variables rather than directly shaping the internal representations learned by neural networks. This limits the ability to analyze how embedded knowledge influences optimization dynamics, convergence behavior, or representation formation. Therefore, NeuPSL is considered informative but not methodologically aligned with the experimental objectives pursued here.

3.3.3 Logic Tensor Networks (LTNs)

Logic Tensor Networks (LTNs)[32] provide a unified framework for combining learning and reasoning. They are built using **Real Logic**, as explored in more detail on 2.5. It allows a principled integration of deductive reasoning over a knowledge base with data-driven relational learning, enabling reasoning over noisy or incomplete information.

When multiple clauses are considered simultaneously, it is not always possible to find a grounding that fully satisfies all constraints, especially in real-world scenarios where data may be incomplete or contradictory. To address this issue, LTNs adopt the notion of approximate satisfiability, in which learning is formulated as the problem of finding a grounding that maximizes the overall satisfaction of the logical theory, rather than strictly enforcing all formulas.

To formalize this setting, LTNs introduce the notions of partial groundings, denoted by $\hat{\mathcal{G}}$, which are groundings defined on a subset of the signature of $\mathcal{L}$, and grounded theories (GTs), which are composed of a set of clauses in the language $\mathcal{L}$ together with a partial grounding $\hat{\mathcal{G}}$. A grounded theory is satisfiable if there exists a grounding $\mathcal{G}$ that extends the partial grounding $\hat{\mathcal{G}}$ such that every clause in the grounded theory is satisfiable under the complete grounding $\mathcal{G}$. A clause is said to be satisfiable with respect to a confidence interval $[v, w]$ if its grounded truth value lies within that interval. If a grounded theory is inconsistent, meaning that no grounding $\mathcal{G}$ satisfies all its clauses, the goal becomes to find a grounding $\mathcal{G}$ that satisfies the grounded theory as much as possible. This is defined as minimizing the satisfiability error, that is, finding a grounding $\mathcal{G}$ that minimizes the total amount by which the clauses violate their respective satisfaction intervals. By restricting the space of groundings to differentiable function families, such as tensor networks, this loss function becomes differentiable with respect to the model parameters. As a result, learning in LTNs can be performed using standard gradient-based optimization techniques. Logical knowledge therefore acts as a structured regularization term that directly shapes the optimization landscape during training. Specific instances

of Real Logic can be obtained by selecting the space $\mathcal{G}$ of groundings and the specific s -norm used for the interpretation of disjunction.

With this encoding, the grounding (i.e., truth value) of a clause is computed by a neural network that first evaluates the groundings of the literals appearing in the clause and then combines them using the selected s -norm. Once predicates and functions are grounded, logical formulas are evaluated by combining the truth values of their constituent literals using the chosen fuzzy operators. Each grounded clause thus yields a differentiable measure of satisfaction in the interval $[0, 1]$.

In practice, this loss is optimized jointly with data-driven objectives, allowing LTNs to balance empirical evidence and logical consistency. As a result, the learned representations are encouraged to satisfy relational constraints while remaining predictive with respect to observed data.

A key motivation for using LTNs in this thesis is their ability to directly incorporate structured, formal knowledge into the learning process. Unlike standard neural networks, which learn purely from data, LTNs allow us to encode logical relationships, ontologies, or domain constraints as differentiable constraints that guide the network’s representations.

Through the grounding mechanism $\mathcal{G}$ and the approximate satisfiability framework, LTNs can:

- Integrate relational knowledge about entities and their attributes directly into the model.
- Transform symbolic rules into differentiable loss terms, which influence the optimization of neural parameters.
- Handle uncertainty and noise in the data while still respecting prior knowledge.

This capability allows us to investigate the impact of pre-imbued knowledge on neural network training. Specifically, we can study:

1. How providing prior knowledge through LTNs affects sample efficiency, and whether fewer training examples are required to reach a given level of performance.
2. How logical constraints influence generalization, robustness to noisy labels, and convergence speed.
3. How different types of relations or subsets of the knowledge base impact learning, by selectively activating parts of the ontology.

By formalizing knowledge as a set of logical clauses and optimizing the network to satisfy them, LTNs provide a controlled setting for understanding how formal knowledge shapes training in NNs.

3.4 Pre-training techniques for Neural-Networks

Pre-training techniques for neural networks aim to initialize models in a way that facilitates more effective learning during the main training phase. Rather than starting from random parameter values, pre-training leverages prior structure, auxiliary tasks, or previously learned representations to provide the network with a more informative starting point. This can improve convergence speed, enhance sample efficiency, and lead to better generalization, particularly in settings where labeled data is limited or training is computationally expensive.

3.4.1 Pre-trained Networks as Implicit Knowledge Sources

The most common approach consists of using a model trained on a related task as the base for a new network, which is then trained on the target task. This technique is called **transfer learning** [35]. It relies on the assumption that NNs, if the tasks are similar, some of the knowledge that was learned from training on a task can be helpful in the other tasks. Beyond standard transfer learning, alternative training strategies leverage pre-trained networks as implicit sources of semantic knowledge without fully adapting all model parameters. In particular, a common variant consists of training only a subset of layers, typically those responsible for higher-level semantic representations, while keeping the remaining layers fixed during the main training phase. This technique, often referred to as **layer freezing** or **frozen feature extraction** [13], aims to preserve previously learned semantic structure while allowing task-specific adaptation in later stages of the network.

Conceptually, this can be viewed as implicit knowledge incorporation. Rather than explicitly encoding formal rules or constraints, semantic information is embedded in the network parameters through pretraining and subsequently preserved by freezing selected layers. As a result, the model is guided by prior knowledge during learning, while still relying on standard optimization procedures.

A further pre-training strategy considered in this work consists of training a smaller NN using data derived directly from the axioms encoded in the ontology, independently of the main task data. In this setting, the network is optimized to satisfy formal constraints or relational properties specified by the axioms, thereby learning representations encoding this knowledge. Once this training phase is completed, the learned parameters are frozen and the subnetwork is integrated into the full model, where it serves as a way to guide the model during task-specific training. This approach is evaluated in the experimental section for its effectiveness and alignment with the thesis objectives.

3.4.2 Impact of Prior Knowledge on Neural Training

The researchers in [31] propose a method for leveraging symbolic knowledge to guide neural training. In contrast with the more commonly used transfer learning, where the weight values from another network trained on a similar dataset are transferred to the new

NN, to be used as initial values, this approach proposes the use of *knowledge prototypes*, that are symbolic representations of the target classes. These prototypes will be used only during pre-training, and there will be a limited amount of them for each class (in the article, the researchers used only one prototype for each class). Once high accuracy on the prototype data is achieved, training proceeds on real data.

This approach is promising when pretrained models are unavailable or real data is scarce, as it can be applied to any classification task where prior knowledge can be converted into data form.

However, despite its relevance, this approach is not adopted in this thesis. The primary reason is that informed pre-training injects knowledge only indirectly, through initialization and auxiliary objectives, making it difficult to isolate and analyze how specific pieces of formal knowledge influence the learning dynamics of the neural network. In contrast, the focus of this work is on approaches that explicitly encode symbolic knowledge as constraints during training, allowing a more direct and interpretable study of the impact of prior knowledge on optimization, generalization, and convergence.

3.5 Concept Mapping

The Concept Mapping framework [34] was developed to improve understanding of how a NN reasons about the input, and how the NN classifies it. Intuitively, it was reasonable to assume that a NN reasons in a similar way as a human would, by identifying key concepts and then identify the respective class. However, due to the complexity and opacity of internal representations of a NN, interpreting the significance of hidden-layer activations is challenging.

Concept Mapping addresses this challenge. The system consists of three parts: the main network that we want to explain, that has already been trained and is not going to be changed by this system; an ontology derived from the dataset; and a collection of auxiliary neural classifiers. This aggregation of smaller classifiers, that are trained to receive part of the activation patterns from the main network and predict the presence of one specific concept of the ontology. When the main network is used, and classifies one new example, the secondary network will read the internal activation patterns from the main network and identify which key concepts were identified by the main network.

The results showed that concepts could be identified without modifying the main network, improving explainability with minimal additional training cost. However, this research has several limitations.

One limitation concerns the dataset used [33], which was not representative of standard image classification benchmarks, restricting generalizability. Another one is that the main NN must be trained using a dataset that contains information about the intermediate concepts that would be identified by the classifiers. Most datasets do not have this data, so this approach could not be applied in most cases. Subsequent work [30] [23] has attempted to mitigate these issues.

Concept Mapping contributes to neural explainability by linking internal representations to human-interpretable concepts. However, as a post-hoc method it does not incorporate formal knowledge into training. Since the main network remains fixed, it cannot support analysis of how symbolic knowledge affects learning dynamics or generalization. Consequently, it falls outside the methodological scope of this thesis.

3.6 Explanations of neural networks with LTNs

The work in [29] was developed in the area of the XAI with the goal of investigating if it was possible to use the LTNs to increase the explainability of neural networks, namely when using CNNs to classify images. More specifically, this work aimed to investigate if it was possible to train the network so it not only would classify the image correctly, but it would also identify the intermediate concepts present in the image based on formal knowledge from the dataset. To achieve this, each intermediate concept defined in the ontology is mapped to a neuron in the penultimate layer of the network, referred to as the *concept layer*. For example, consider the following axiom incorporated into the model:

$$\text{Cafe} \equiv \exists \text{hasStatue} \sqcup \exists \text{hasVendingMachine}$$

In this example, the intermediate concepts are *Statue* and *VendingMachine*. The logical connectives are implemented as continuous functions, and the predicates are grounded using the corresponding neuron outputs. This formulation enables the computation of a degree of satisfaction for each axiom, which is integrated into the loss function and used to guide the optimization of the model during training.

The objective is that, whenever the model would correctly predicts that the image depicted a cafe, it would also correctly identify that the concepts *Statue* and *VendingMachine* were also present in the image by assigning high truth values to the correspondent neurons. In this example, the concept layer would map each of the aforementioned concepts to one of the neurons present in this layer. Logical axioms encoding domain knowledge are then incorporated into the training process by translating them into differentiable fuzzy logic expressions.

Using the same example, the truth value of the disjunctive expression at the end of a training iteration can be computed using the probabilistic t-conorm. Let a and b denote the truth values assigned by the network to the neurons representing the concepts *Statue* and *VendingMachine* for a given sample, with $a, b \in [0, 1]$. The disjunction is then evaluated as

$$a \sqcup b = a + b - ab.$$

For instance, if the network predicts $a = 0.7$ and $b = 0.4$, the resulting truth value of the formula is $0.7 + 0.4 - (0.7 \cdot 0.4) = 0.82$. This value represents the degree to which the sample satisfies the logical constraint and is directly incorporated into the loss function, where higher satisfaction values correspond to lower penalties during optimization.

3.6.1 Model Architecture

The architecture proposed in this work is based on a convolutional neural network designed to support the explicit representation of intermediate semantic concepts.

A distinguishing component of the architecture is the *concept layer*, which is an intermediate dense layer that is placed immediately before the final classification layer. Each neuron in this layer is associated with a predefined semantic concept chosen from the ontology. The activations of the concept layer are then used as inputs to the final output layer, which produces the class predictions.

This design allows the model to output both the predicted class labels and the activations of the intermediate concepts. These concept activations are subsequently used to ground logical predicates within the LTN framework, enabling the incorporation of formal knowledge through logical constraints during training.

3.6.2 Structure of the Model Output

To enable the integration between the CNN and the LTN framework, the model's output is structured in a unified format compatible with the LTN predicates. Since the LTN framework supports only a single output layer as input to each predicate, the CNN is designed to concatenate both the final class predictions and the intermediate concept activations into a single output vector.

In this structure, the first subset of neurons corresponds to the target classification labels, while the remaining neurons represent the intermediate semantic concepts defined in the ontology. Each concept neuron encodes the presence of a specific concept in the input sample, with values close to 1 indicating presence and values close to 0 indicating absence. This structure is illustrated in Figure 3.2, reproduced from [29].

Importantly, no activation function is applied to the neurons corresponding to the concept layer at the CNN level. This design choice ensures that the outputs are provided to the LTN model in the form of logits, as required by the LTN framework. The LTN predicates subsequently apply a sigmoid activation function internally to convert these logits into truth values within the $[0, 1]$ interval.

This output formulation allows the LTN model to directly ground logical predicates on both the predicted class labels and the intermediate concepts, enabling the enforcement of logical constraints and the extraction of concept-level explanations during training.

3.6.3 Training with Logical Constraints

Training the neural-symbolic model follows a standard gradient-based optimization procedure, but differs from conventional CNN training in the composition of the loss function. In addition to the task-specific loss used for image classification, the model incorporates a logic-based loss term derived from the satisfaction of the logical axioms encoded in the ontology. Each axiom is translated into a differentiable expression whose

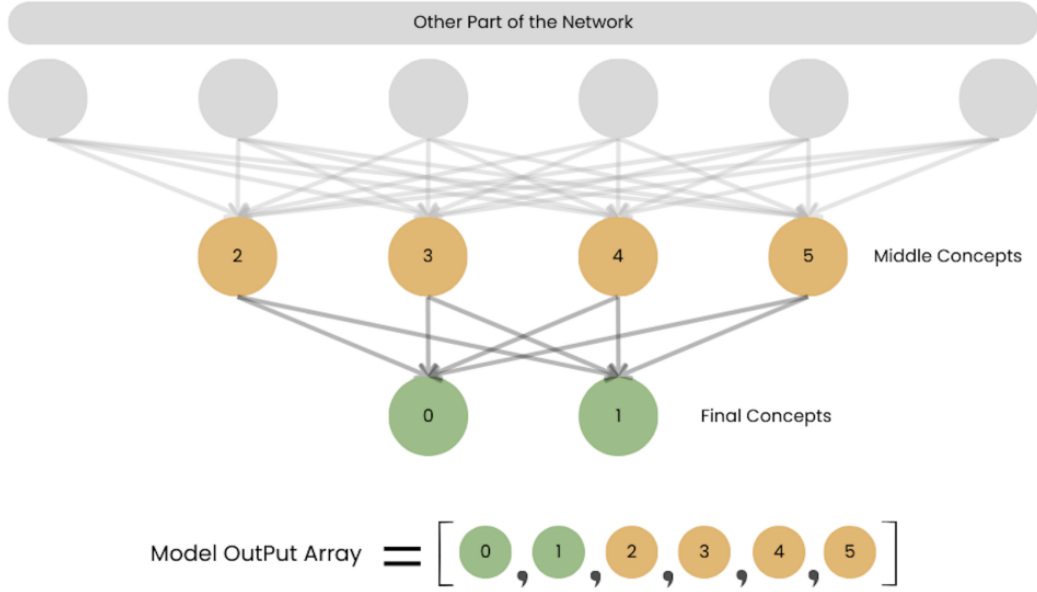


Figure 3.2: Model output: each number inside the circle indicates the position of the neuron in the result array passed to the predicate. The first two neurons represent the final class of the image, while the last four correspond to intermediate concepts of the dataset. Reproduced from [29].

truth value reflects the degree to which the model’s predictions comply with the formal knowledge.

The overall training objective is defined as a weighted combination of the classification loss and the aggregated logic satisfaction loss. This formulation treats logical constraints as soft regularizers rather than hard requirements, allowing the model to balance data-driven learning with prior symbolic knowledge. The weighting factor controls the relative influence of the logical component, enabling the model to prioritize predictive accuracy or logical consistency depending on the task requirements.

3.6.4 Evaluation methodology

After executing the experiments, the researchers measure the results using two different techniques: occlusion maps [39] and confusion matrices [9].

Occlusion maps is a technique used to understand how a CNN generates its results. It determines which regions of the image are the most influential to the precision of the predictions by systematically changing the input images, and then measuring the results. In this work, occlusion maps were used to verify whether the regions identified by the model as most relevant correspond to the intermediate concepts introduced through formal knowledge.

Confusion matrices are used to measure the predictive performance of classification models, by comparing the model’s prediction labels with the corresponding true results.

They summarize the number of instances classified correctly and incorrectly, organizing the results into four categories: true positives, true negatives, false positives, and false negatives. This representation provides a detailed view of the types of errors made by the model, going beyond overall accuracy and enabling a more fine-grained analysis of class-wise performance and misclassification patterns.

The authors evaluate the effectiveness of their neural-symbolic approach through a combination of quantitative and qualitative analyses, aiming to assess both predictive performance and semantic consistency. Model performance is measured not only in terms of classification accuracy but also through the satisfiability of the logical knowledge base, which reflects how well the learned predictions comply with the imposed axioms. Satisfiability values are monitored throughout training and testing to analyze the alignment between neural outputs and logical constraints. To better understand the behavior of intermediate concepts introduced through logical axioms, the authors report accuracy scores for both final classes and concept-level outputs across training epochs. Since intermediate concepts are not directly supervised by labels, their performance is evaluated indirectly through their contribution to axiom satisfaction. This allows the authors to study how logical constraints influence the learning of latent concepts.

In addition, confusion matrices are employed to analyze error patterns and concept interactions, particularly in scenarios involving conjunctions and disjunctions. By comparing confusion matrices under different activation conditions, the authors investigate whether the model learns distinct representations for each concept or collapses multiple concepts into a single latent representation. Finally, the evaluation is complemented with occlusion map visualizations, which provide qualitative insights into the spatial regions of the input images that influence the activation of specific concepts. These visualizations are used to verify whether intermediate concepts correspond to meaningful visual patterns and to diagnose cases where disjunctive concepts are incorrectly unified. Together, these evaluation techniques provide a comprehensive assessment of both predictive accuracy and semantic interpretability.

3.7 Summary and Method Comparison

In this chapter we review a range of approaches for integrating prior knowledge into neural network learning, including pre-training strategies, logic-based regularization methods, and neural-symbolic frameworks. While all these approaches aim to improve learning efficiency or interpretability, they differ fundamentally in how and where symbolic knowledge influences the model.

Pre-training and knowledge-guided initialization inject prior knowledge only implicitly, through parameter initialization or auxiliary objectives, making it difficult to isolate their impact on training dynamics. Similarly, post-hoc explanation techniques such as Concept Mapping improve interpretability but do not influence the learning process itself.

Logic regularization approaches, such as Semantic Loss, introduce formal constraints directly into the training objective but operate exclusively at the output level, limiting their influence on internal representations. NeuPSL provides a tighter integration between neural and symbolic components, but its symbolic knowledge primarily affects inference over output variables rather than shaping neural representations during training.

In contrast, LTNs enable the explicit incorporation of formal knowledge as differentiable constraints that directly influence neural parameters throughout training. By grounding logical predicates in intermediate representations and optimizing for approximate satisfiability, they provide a controlled and interpretable mechanism for studying how prior knowledge affects optimization, generalization, and convergence.

METHODOLOGY

In this chapter we describe the approach that we will take to achieve the goals outlined earlier, as well as other additional methodological details for the experimentation phase such as the dataset, model architecture, evaluation metrics, planned experiments, and finally a first plan of the following months.

4.1 Goals and approach

As a starting point in the development of our models, we adopt the same architecture proposed in this work [29]. This choice is motivated by its demonstrated success in prior work involving LTNs. The architecture of the system that they propose integrates LTNs into a CNN and will be described in more detail later. Using the LTN framework, we define axioms and relationships between the final classes and the intermediate concepts that will guide the network during training. Within the LTN model, predicates are defined to evaluate logical relations. These predicates compute truth values for classes and intermediate concepts, which are then combined to evaluate axioms. By applying a formula aggregator to these values, we can compute a satisfiability score for all the axioms defined, which contributes to the CNN’s loss function. With each training iteration, this satisfiability score improves gradually, epoch by epoch. In this way, the CNN and LTN work together to enforce learning through logical constraints and predefined axioms.

After conducting the simpler experiments, with simpler subsets of ontology expressions, we will investigate the feasibility of using a pre-training phase to incorporate the logical formulas into the network instead of LTNs.

4.2 Dataset Selection

We will begin by analyzing the VCB dataset, developed in [30], to assess its suitability as the main dataset for the experimental phase of this thesis. This choice is motivated by two main factors. First, the VCB dataset has already been used in previous work in this research area [29] [23], where it was shown to have sufficient expressiveness for learning-based

and neural-symbolic methods. This makes it a reliable and well-characterized benchmark for evaluating the proposed approach.

Second, the VCB dataset is accompanied by a rich and well-structured ontology. Since a central objective of this thesis is to study how formal knowledge can be integrated into neural learning, the availability of an expressive ontology is crucial. It enables the formulation of meaningful logical constraints over the data and supports a principled evaluation of how symbolic knowledge influences training and generalization.

If we deem the dataset to be appropriate, we will then analyze the ontology and, if required, propose minor modifications to ensure that it can express adequately the constraints and concepts required by the planned experimental setup.

4.3 Architecture

The neural architecture adopted in this thesis is a CNN designed to support the integration of formal knowledge through LTNs. The model follows a standard deep learning pipeline for image classification, while explicitly exposing internal representations that can be associated with symbolic concepts.

The network begins with a sequence of convolutional layers interleaved with max-pooling operations, which are responsible for extracting hierarchical visual features and reducing spatial dimensionality. The resulting feature maps are then flattened and passed through fully connected layers, with dropout applied to mitigate overfitting.

Crucially, the final part of the network is structured into two dense layers. The concept layer, as mentioned before, is designed in such a way that each intermediate concept defined in the ontology has a correspondent neuron in this layer. The goal of these neurons is to capture the visual patterns associated with the concepts from the ontology, such as objects or attributes present in the image. The last fully-connected layer is the output layer, and is responsible for the final classification of the image. In this layer, each neuron represents a target class.

The integration of LTNs occurs during training through the definition of the loss function. The activations of the neurons in the concept layer are interpreted as predicates representing the degree of truth of the corresponding concepts. These activation are the interface between the logical knowledge that was encoded and their neural representation.

Using the LTN framework, we can translate the logical axioms from the ontology into differentiable constraints. Each formula is evaluated through fuzzy logic, and produces a satisfiability score that measures how much the network’s predictions respect to the formal knowledge. These scores are then aggregated into a symbolic term, that will be combined with the conventional loss function of a CNN.

Formally, the total loss can be expressed as a weighted sum of the CNN’s loss function and the logical satisfiability loss, encouraging the network to not only perform accurate predictions but also remain consistent with domain knowledge. This allows symbolic

reasoning to guide representation learning without altering the forward architecture of the CNN itself.

4.4 Evaluation metrics

To evaluate the performance of the proposed neural-symbolic models, we adopt a combination of standard predictive metrics and knowledge-aware metrics. This choice reflects the dual objective of this thesis: not only to achieve accurate visual classification, but also to assess how well the learned models comply with the formal knowledge encoded in the ontology and logical constraints. For the evaluation of our work we will use the following metrics, that were introduced in [2.6.1](#).

- Classification Accuracy and F1-Score.
- Sample Efficiency
- Training Dynamics
- Constraint Satisfaction.
- Semantic Consistency

4.5 Experiments

In this section we describe the experimental design adopted to evaluate the impact of integrating formal knowledge into neural network training. All experiments are conducted under controlled conditions, ensuring that knowledge-informed models and baseline models share the same neural architecture and data splits. This allows us to isolate the effects of formal knowledge integration and perform fair comparisons across models.

As a reference point, we first evaluate a baseline neural network trained without formal knowledge. This model relies solely on data-driven supervision and is evaluated using standard predictive metrics, including classification accuracy, F1-score, and training dynamics such as convergence speed. These results establish the expected performance of the architecture when it is trained without formal knowledge.

To assess the overall impact of formal knowledge, we train a neural-symbolic model using LTNs, where logical constraints derived from the ontology are incorporated into the training objective as soft constraints. Its performance is evaluated using the same metrics and compared against the baseline to determine whether knowledge integration improves learning outcomes beyond explainability.

We further examine data efficiency through a sample efficiency study in which both models are trained on progressively smaller subsets of the training data. Performance trends across subset sizes are examined to determine whether formal knowledge enables comparable results with fewer examples.

Training dynamics are analyzed by tracking performance across epochs and measuring convergence speed as the number of epochs required to reach fixed performance thresholds. Comparing learning curves provides insight into stability, convergence behavior, and sensitivity to initialization.

To investigate the trade-off between optimization and logical consistency, we vary the strength with which logical constraints are enforced in the LTN framework, ranging from weak to strong regularization. Models are compared in terms of predictive performance, convergence speed, and constraint satisfaction to identify a balance between efficient learning and logical reliability.

Finally, we study the effect of constraint complexity by progressively introducing richer logical relationships, from simple conjunctions and disjunctions to more structured combinations. Each configuration is evaluated using predictive and knowledge-aware metrics, allowing us to analyze how increasingly expressive symbolic structure influences learning behavior.

4.6 Task Schedule

Table 4.1 summarizes the planned timeline for the development, experimentation, and evaluation phases of this thesis. The experimental work begins with the implementation of a baseline convolutional neural network, followed by the integration of formal knowledge using LTNs as an initial NeSy framework. These first experimental phases serve both as a reference point and as an empirical foundation for assessing the impact of explicit knowledge integration during training.

Following the initial experiments, dedicated investigation and decision phases are planned to analyze the accumulated results and to assess the suitability of alternative or complementary techniques for incorporating formal knowledge into neural models. Based on the outcome of this analysis, subsequent experimental phases will be conducted using the selected approach. Each cycle allocates approximately three weeks to model integration, training, and testing, followed by a dedicated evaluation phase. This structure was chosen to ensure sufficient time for systematic experimentation while maintaining regular checkpoints for analysis and methodological adjustment. The number of phases reflects the anticipated iterations required to explore increasingly complex configurations of knowledge integration, rather than representing arbitrary divisions of the timeline.

The final months are reserved for global evaluation and result consolidation, ensuring that all experimental findings are thoroughly analyzed and clearly reported before submission. Although writing progresses throughout the research process, the majority of thesis preparation is scheduled for September.

Phase	Task	Period
Design	Dataset, ontology, tests	Late Feb – Mid Mar
Baseline CNN	Architecture + baseline	Mid Mar – Early Apr
Experiment phase 1	Model integration and testing	Apr – Late Apr
Evaluation of phase 1	Results analysis	Late Apr
Experiment phase 2	Model integration and testing	May – Late May
Evaluation of phase 2	Results analysis	Late May
Investigation	Evaluating all previous results	Early Jun
Decision	Pre-training decision	Mid Jun
Experiment phase 3	Model integration and testing	Mid Jun – Early July
Evaluation phase 3	Results analysis	Mid July
Experiment phase 4	Model integration and testing	Mid July – Early Aug
Evaluation phase 4	Results analysis	Mid Aug
Evaluation	Global analysis	Mid Aug – Early Sep
Thesis Preparation	Writing	Throughout project (Sep focus)

Table 4.1: Planned thesis timeline

BIBLIOGRAPHY

- [1] H. Abdi, D. Valentin, and B. Edelman. *Neural networks*. 124. Sage, 1999 (cit. on pp. 1, 6).
- [2] E. Alpaydin. *Machine learning*. MIT press, 2021 (cit. on p. 5).
- [3] F. Baader et al. *An introduction to description logic*. Cambridge University Press, 2017 (cit. on p. 9).
- [4] B. P. Bhuyan et al. “Neuro-symbolic artificial intelligence: a survey”. In: *Neural Computing and Applications* 36.21 (2024), pp. 12809–12844 (cit. on p. 2).
- [5] M. Brocheler, L. Mihalkova, and L. Getoor. *Probabilistic Similarity Logic*. 2012. arXiv: [1203.3469 \[cs.AI\]](#) (cit. on pp. 16, 20).
- [6] R. Chandana and A. Ramachandra. “Real time object detection system with YOLO and CNN models: A review”. In: *arXiv Prepr. arXiv2208 773* (2022) (cit. on p. 9).
- [7] T. Developers. “TensorFlow”. In: *Zenodo* (2022) (cit. on p. 5).
- [8] M. R. Douglas. “Large language models”. In: *arXiv preprint arXiv:2307.05782* (2023) (cit. on p. 1).
- [9] J. Erbani et al. “Confusion Matrices: A Unified Theory”. In: *IEEE Access* 12 (2024), pp. 181372–181419. DOI: [10.1109/ACCESS.2024.3507199](#) (cit. on p. 27).
- [10] M. Greenacre et al. “Principal component analysis”. In: *Nature Reviews Methods Primers* 2.1 (2022), p. 100 (cit. on p. 6).
- [11] K. Hacıfendioglu et al. “CAM-K: a novel framework for automated estimating pixel area using K-Means algorithm integrated with deep learning based-CAM visualization techniques”. In: *Neural Computing and Applications* 34.20 (2022), pp. 17741–17759 (cit. on p. 6).
- [12] B. Hambly, R. Xu, and H. Yang. “Recent advances in reinforcement learning in finance”. In: *Mathematical Finance* 33.3 (2023), pp. 437–503 (cit. on p. 6).
- [13] L. Hertel et al. “Deep Convolutional Neural Networks as Generic Feature Extractors”. In: (2017-10). DOI: [10.48550/arXiv.1710.02286](#) (cit. on p. 23).

- [14] S. Imambi, K. B. Prakash, and G. Kanagachidambaresan. “PyTorch”. In: *Programming with TensorFlow: solution for edge computing applications*. Springer, 2021, pp. 87–104 (cit. on p. 5).
- [15] N. Ketkar and J. Moolayil. “Convolutional neural networks”. In: *Deep learning with Python: learn best practices of deep learning models with PyTorch*. Springer, 2021, pp. 197–242 (cit. on p. 8).
- [16] D. Koller et al. *Introduction to Statistical Relational Learning*. MIT Press, 2007 (cit. on p. 16).
- [17] N. Lavrac and S. Dzeroski. “Inductive logic programming.” In: *Wlp*. Springer. 1994, pp. 146–160 (cit. on p. 16).
- [18] J. Leng et al. “A multi-objective reinforcement learning approach for resequencing scheduling problems in automotive manufacturing systems”. In: *International Journal of Production Research* 61.15 (2023), pp. 5156–5175 (cit. on p. 6).
- [19] R. Manhaeve et al. “Deepproblog: Neural probabilistic logic programming”. In: *Advances in neural information processing systems* 31 (2018) (cit. on pp. 17, 18).
- [20] G. Marcus. “The Next Decade in AI: Four Steps Towards Robust Artificial Intelligence”. In: *CoRR abs/2002.06177* (2020). arXiv: [2002.06177](https://arxiv.org/abs/2002.06177) (cit. on p. 1).
- [21] K. F. Mojdehi, B. Amiri, and A. Haddadi. “A novel hybrid model for credit risk assessment of supply chain finance based on topological data analysis and graph neural network”. In: *IEEE Access* (2025) (cit. on p. 1).
- [22] L. Morissette, S. Chartier, et al. “The k-means clustering technique: General considerations and implementation in Mathematica”. In: *Tutorials in Quantitative Methods for Psychology* 9.1 (2013), pp. 15–24 (cit. on p. 6).
- [23] R. B. PATRONILO. “JUSTIFYING THE OUTPUT OF NEURAL NETWORKS BY GUIDING THE TRAINING WITH ONTOLOGIES”. MA thesis. Universidade NOVA, 2025 (cit. on pp. 24, 30).
- [24] D. Poole. “Logic, probability and computation: Foundations and issues of statistical relational AI”. In: *International Conference on Logic Programming and Nonmonotonic Reasoning*. Springer. 2011, pp. 1–9 (cit. on p. 17).
- [25] S. P. Praveen et al. “AI-powered diagnosis: Revolutionizing healthcare with neural networks”. In: *Journal of Theoretical and Applied Information Technology* 103.3 (2025), pp. 982–990 (cit. on p. 1).
- [26] C. Pryor et al. *NeuPSL: Neural Probabilistic Soft Logic*. 2023. URL: <https://arxiv.org/abs/2205.14268> (cit. on p. 20).
- [27] H. Qiu et al. “Integrating digital twins with neural networks for adaptive control of automotive suspension systems”. In: *Scientific Reports* 15.1 (2025), p. 11078 (cit. on p. 1).

- [28] L. D. Raedt et al. "From Statistical Relational to Neuro-Symbolic Artificial Intelligence". In: *CoRR abs/2003.08316* (2020). arXiv: [2003.08316](#) (cit. on p. 17).
- [29] B. M. REIS. "EXPLANATIONS OF NEURAL NETWORKS WITH LOGIC TENSOR NETWORKS". MA thesis. Universidade NOVA, 2024 (cit. on pp. 2, 25–27, 30).
- [30] B. A. C. RODRIGUES. "DESIGNING CONCEPT MAPPING MODELS WITH NEURAL ARCHITECTURE SEARCH". MA thesis. Universidade NOVA, 2022 (cit. on pp. 3, 24, 30).
- [31] L. von Rueden et al. *Informed Pre-Training on Prior Knowledge*. 2022. arXiv: [2205.11433](#) [cs.LG] (cit. on p. 23).
- [32] L. Serafini and A. d'Avila Garcez. *Logic Tensor Networks: Deep Learning and Logical Reasoning from Data and Knowledge*. 2016. arXiv: [1606.04422](#) [cs.AI] (cit. on pp. 2, 3, 21).
- [33] M. de Sousa Ribeiro, L. Krippahl, and J. Leite. "Explainable abstract trains dataset". In: *CoRR abs/2012.12115* (2020) (cit. on p. 24).
- [34] M. de Sousa Ribeiro and J. Leite. "Aligning Artificial Neural Networks and Ontologies towards Explainable AI". In: *Proceedings of the AAAI Conference on Artificial Intelligence* 35.6 (2021), pp. 4932–4940. DOI: [10.1609/aaai.v35i6.16626](#) (cit. on p. 24).
- [35] L. Torrey and J. Shavlik. "Transfer learning". In: *Handbook of research on machine learning applications and trends: algorithms, methods, and techniques*. IGI Global Scientific Publishing, 2010, pp. 242–264 (cit. on p. 23).
- [36] T. M. Usman et al. "Diabetic retinopathy detection using principal component analysis multi-label feature extraction and classification". In: *International Journal of Cognitive Computing in Engineering* 4 (2023), pp. 78–88 (cit. on p. 6).
- [37] J. Xu et al. *A Semantic Loss Function for Deep Learning with Symbolic Knowledge*. 2018. arXiv: [1711.11157](#) [cs.AI] (cit. on p. 19).
- [38] F. Yuan, Z. Zhang, and Z. Fang. "An effective CNN and Transformer complementary network for medical image segmentation". In: *Pattern Recognition* 136 (2023), p. 109228 (cit. on p. 9).
- [39] M. D. Zeiler and R. Fergus. *Visualizing and Understanding Convolutional Networks*. 2013. arXiv: [1311.2901](#) [cs.CV] (cit. on p. 27).
- [40] A. Zhang et al. *Dive into Deep Learning*. Cambridge University Press, 2023. URL: <https://d2l.ai> (cit. on p. 6).
- [41] W. Zhou, H. Wang, and Z. Wan. "Ore image classification based on improved CNN". In: *Computers and Electrical Engineering* 99 (2022), p. 107819 (cit. on p. 9).

